

# SimpleSave - Unity Save System Documentation

## Introduction

SimpleSave is a lightweight and flexible Unity asset that makes saving and loading user data simple and powerful.

Whether you're working with primitive types, custom data structures, or Unity components like Transform, SimpleSave gives you a unified and type-safe API.

## How to Save & Load Data

Saving data:

```
SimpleSaver.Save("key", data);
```

Loading data:

```
var loaded = SimpleSaver.Load<Type>("key");
```

## Supported Data Types

Basic types: int, float, string, bool

Unity types: Vector2/3/4, Quaternion, Color, Transform

Collections: List<T>, Dictionary<K, V>, Queue<T>, HashSet<T>

Custom classes with [Serializable] attribute

## Saving Custom Classes

To save your own class, it must be marked with [Serializable]:

[Serializable]

```
public class PlayerData {  
    public int level;  
    public float health;  
    public Vector3 position;  
}
```

If your class contains a Dictionary, it must be converted to a SerializableDictionary!

[Serializable]

```
public class MyData {  
    public SerializableDictionary<string, int> stats;  
}
```

## Code Examples

// Save and Load simple type

```
SimpleSaver.Save("score", 100);  
  
int score = SimpleSaver.Load<int>("score");
```

// Save and Load a custom class

```
var player = new PlayerData { level = 10, health = 99f };  
  
SimpleSaver.Save("player", player);  
  
var loaded = SimpleSaver.Load<PlayerData>("player");
```

// Save and Load a list

```
List<string> names = new() { "Alex", "Jamie" };  
  
SimpleSaver.Save("names", names);  
  
var loadedNames = SimpleSaver.Load<List<string>>("names");
```

// Save and Load a Transform

```
SimpleSaver.Save("heroTransform", transform);  
  
SimpleSaver.LoadInto("heroTransform", transform);
```

## Encryption

You can enable encryption in the Unity Editor:

1. Open the SimpleSave Manager: Unity Menu > SimpleSave > Manager
2. Enable 'Use Encryption'
3. Set a secure password

Encrypted files use AES for security.

## Format Options

You can choose how data is stored:

- Binary (.bin) - compact and fast
- JSON (.json) - human-readable
- PlayerPrefs - key-value store for simple use cases

## Editor Tools

Open the SimpleSave Manager via:

Unity Menu > SimpleSave > Manager

Features include:

- File viewer
- Delete/refresh options
- Settings configuration

## Async Support

SimpleSave supports asynchronous operations:

```
await SimpleSaver.SaveAsync("key", data);  
var result = await SimpleSaver.LoadAsync<Type>("key");
```

## Conclusion

SimpleSave makes it easy to implement robust save systems in Unity.

- Mark your classes with [Serializable]
- Use SerializableDictionary when working with Dictionary

Enjoy developing with SimpleSave and happy coding!

## Useful Links

Unity Asset Store:

<https://assetstore.unity.com/publishers/118904?preview=1>

Support on Ko-fi:

<https://ko-fi.com/pandapps>